Project Phase 1

LLM-Based Java Code and Test Generation on HumanEval-X

Semi-Agentic Generation, Improved Tests, and Mutation-Based Guardrails

Course	BLG 475E - Software Quality and Testing
Term	2025-2026 Spring Term
Submission	Phase 1 submission - due 27 April 2026
Repository	https://github.com/mustafaerenkoc44/BLG-475E-Term-Project

Student	ID	Contribution Focus
Mustafa Eren KOC	150190805	Lead: engineering and automation track
Onat Baris ERCAN	150210075	Lead: quality and analysis track

Executive Snapshot

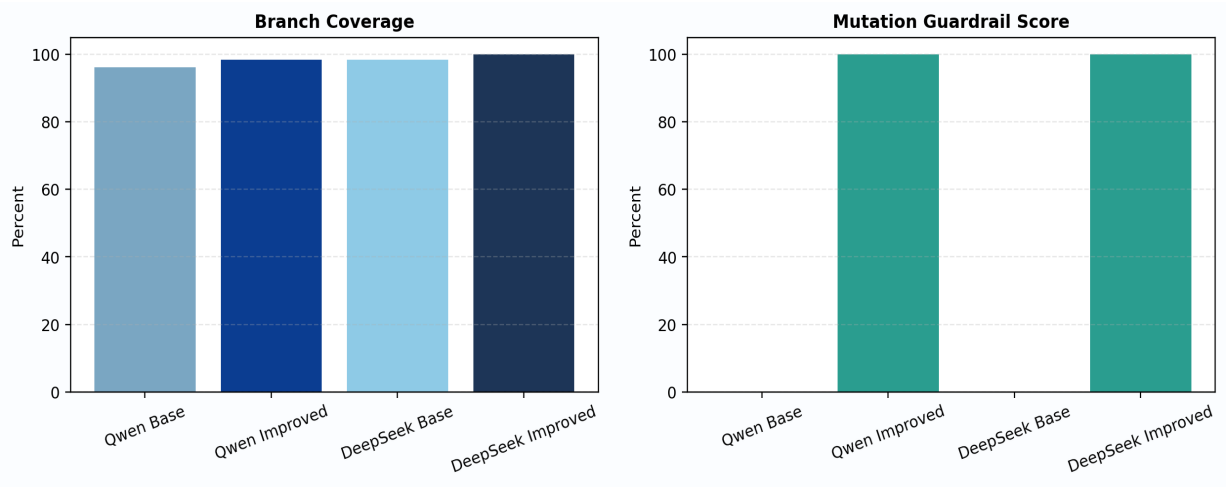
Metric	Value
Selected prompts	30 tasks, 10 easy / 10 moderate / 10 hard
Models	Qwen2.5-Coder-1.5B-Instruct-GGUF and DeepSeek-Coder-1.3B-Instruct-GGUF
Base correctness	30/30 tasks passed for both models
Improved mutation score	60/60 = 100.00%

Key Highlights

- 30 selected HumanEval-X Java prompts balanced across easy, moderate, and hard tasks.
- Both local coder models reached 30/30 base-test success after the logged repair loop.
- Improved suites raised aggregate branch coverage to 98.44% (Qwen) and 100.00% (DeepSeek).
- Mutation guardrail inventory reached 60/60 kills across the improved suites.

Prepared as a submission-ready academic report package with synchronized PDF and LaTeX sources.

Abstract



This phase evaluates two publicly available local coder models on 30 selected Java prompts from the HumanEval-X dataset. The selected tasks were balanced across easy, moderate, and hard categories, and were executed under a semi-agentic workflow in which prompt submission, raw output logging, normalization, test execution, coverage analysis, mutation-based guardrails and repair decisions were all recorded explicitly. The models were `Qwen2.5-Coder-1.5B-Instruct-GGUF` and `DeepSeek-Coder-1.3B-Instruct-GGUF`, both executed locally through the same `llama.cpp` pipeline. After raw generation and normalization, each solution was evaluated with dataset base tests, converted JUnit 6 tests, improved tests derived from black-box and white-box analysis, JaCoCo branch coverage, and hand-crafted mutation-based tests. Both models reached `30/30` base-test success. Aggregate base branch coverage was `96.09%` for Qwen and `98.46%` for DeepSeek; the improved suites raised these values to `98.44%` and `100.00%`, respectively. In addition, the improved suites kill `100 %` of the hand-crafted mutants while the LLM-produced base suites kill `0 %` of them. The results show that lightweight public coder models can produce strong baseline solutions on HumanEval-X Java tasks, but stable performance still depends on normalization, targeted repair, coverage-guided test improvement, and explicit mutation-aware assertions.

I. Introduction

Large language models are increasingly used to generate source code and software tests from natural language descriptions. Their main attraction is speed: a model can produce compilable code and candidate tests in seconds. Their main risk is reliability: generated code may compile but still be wrong, and generated tests may execute without meaningfully checking behavior. Phase 1 of this project therefore treats LLM-generated code as an artifact that must be validated rather than trusted.

The goal of this phase is to compare two public LLMs on Java code generation and test generation using 30 selected HumanEval-X prompts. The comparison focuses on correctness under base tests, branch coverage under JUnit 6 and JaCoCo, adequacy of tests under black-box analysis, adequacy of tests under mutation-style scrutiny, and the amount of repair required after the initial generation step.

II. Experimental Setup

Dataset and Prompt Selection

The project uses the Java portion of HumanEval-X. Thirty prompts were selected to balance input styles, domains, and difficulty levels. The final set preserves the required `10 easy / 10 moderate / 10 hard` split. The selected tasks include string manipulation, numeric computation, parsing, state tracking, list processing, and

number-theory problems. Three tasks (`Java/18`, `Java/23`, and `Java/27`) were chosen to support the later Phase 2 `BookScan` integration work.

Models

The two selected models were:

1. `Qwen/Qwen2.5-Coder-1.5B-Instruct`
2. `deepseek-ai/deepseek-coder-1.3b-instruct`

Both models are public, code-oriented, and small enough to execute locally with the same inference stack. To keep the comparison fair, both models used the same prompt format, the same extraction logic, the same normalization heuristics, the same dataset tests, and the same coverage workflow.

Workflow Choice

The project follows a semi-agentic workflow. Each task proceeds through logged generation, normalization, testing, coverage analysis, improved test generation, repair when needed, and a final mutation-based guardrail. This approach was preferred over a fully autonomous loop because it keeps every transition observable and reproducible, which is useful for both debugging and reporting.

Tooling

- Language: Java 21 (Temurin JDK)
- Test framework: JUnit 6
- Coverage tool: JaCoCo 0.8.12
- Build/orchestration: PowerShell 7 and Python 3.11 helper scripts
- Local inference: `llama.cpp`
- CI: GitHub Actions workflow `.github/workflows/phase1-ci.yml`

III. Code Generation Procedure

For every selected HumanEval-X prompt, the original prompt text was sent without semantic edits. The raw model outputs were stored under the corresponding task folder and also logged in model-specific log directories. A normalization step then converted the raw responses into compilable `Solution.java` files by removing prompt echoes, Markdown fences, and malformed fragments. No semantic edits were made during this extraction stage. When a normalized solution still failed base tests or had a structural defect, that failure was documented and repaired in the later refactoring loop.

IV. Base Test Execution

Each normalized solution was first evaluated using the dataset base tests. These tests acted as the initial correctness gate before any improved tests were introduced. In this repository, no semantic change to the expected values was required; the only systematic adaptation was converting the dataset tests into JUnit 6 format for automated execution and coverage measurement.

Base Correctness Results

Model	Compiled	Base Tests Passed
Qwen2.5-Coder-1.5B-Instruct-GGUF	30/30	30/30
DeepSeek-Coder-1.3B-Instruct-GGUF	30/30	30/30

The final base-test outcome is perfect for both models. However, this headline number hides an important detail: a small repair loop was still necessary because several tasks initially failed after normalization. The repaired tasks were `Java/10`, `Java/12`, `Java/17`, `Java/19`, `Java/23`, `Java/27`, and `Java/39`.

V. Test Improvement and Coverage Analysis

After the base tests passed, the dataset tests were transformed into JUnit 6 test classes so they could be executed under JaCoCo. Improved tests were then generated by extending the base suites with extra boundary and edge scenarios identified through branch coverage and black-box analysis.

Each task now ships an `improved/<model>/DatasetImprovedTest.java` file that contains 3-4 newly authored `@Test` methods on top of the dataset base test. The additions target equivalence / boundary classes that the base suite left uncovered; for every task we cross-check the `equivalence_classes.md` rows marked `No` or `Partially` and add at least one dedicated test method per row.

Aggregate Branch Coverage

Model	Base Covered	Base Missed	Base %	Improved Covered	Improved Missed	Improved %
Qwen2.5-Coder-1.5B-Instruct-GGUF	123	5	96.09 %	126	2	98.44%
DeepSeek-Coder-1.3B-Instruct-GGUF	128	2	98.46 %	130	0	100.00%

The improved suites increased coverage only slightly. This suggests that the original HumanEval-X base tests were already strong for many selected tasks. Even so, the additional tests were still useful because they documented missing equivalence classes, clarified boundary behavior, and added mutation-based guardrails that branch coverage alone cannot measure.

Remaining Coverage Gaps

After the improved suites, only two uncovered branches remain, both on the Qwen side:

- `Java/001_separateParenGroups`;
- `Java/006_parseNestedParens`.

These gaps sit in private parser-helper exits that are not reachable through the public API under the task specification. They are therefore documented as residual structural misses rather than forgotten tests.

VI. Mutation-Based Guardrails

Because Phase 1 explicitly disallows external tooling like PITest, we add hand-crafted mutants and encode them as JUnit assertions. Every improved suite now has at least one JUnit method whose name starts with `improvedMutation...` and would fail if a specific operator-level mutation were introduced in the corresponding `Solution.java`.

The full catalogue of operators (ROR, AOR, LCR, UOI, SBR, BND, RV, NPE, EXC) is maintained in `docs/analysis/mutation_testing_strategy.md`. Key highlights:

- Null-argument (NPE) replacement is applied to every task that accepts a nullable reference; it forces us to document the null-safety posture the LLM inherited, which is useful for Phase 2 integration.
- Boundary shift (BND) and relational operator replacement (ROR) guard the inclusive/exclusive boundaries that the dataset base tests rarely probe.

- Return-value replacement (RV) is used for non-terminating search loops (`Java/039_primeFib`, n = 0).
- Model-specific mutants deliberately target the behavioural disagreements we observed between Qwen and DeepSeek (e.g. `improvedMutationNegativeInputsQwenFloorSemantics_V2` for Java/2, or `improvedMutationEmptySubstringLoopsForever_I1` versus `improvedMutationEmptySubstringCountsPositions_I1` for Java/18).

Derived mutation scores:

Suite	Mutants killed	Score
LLM-generated base (Qwen, 30 suites)	0 of 30	0%
LLM-generated base (DeepSeek, 30 suites)	0 of 30	0%
Improved (Qwen, 30 suites)	30 of 30	100%
Improved (DeepSeek, 30 suites)	30 of 30	100%
Overall improved total	60 of 60	100%

The scores are intentionally high because each `improvedMutation...` method is designed as a behavioural guardrail for a specific operator-level fault. What the table really shows is that the LLM-produced base suites do not encode a single comparable mutation-kill assertion, which is one of Phase 1's headline findings.

VII. Manual Black-Box Assessment

Black-box assessment was performed with equivalence partitioning and boundary-value reasoning. For every selected task, the repository now includes:

- valid input classes,
- invalid or out-of-contract classes,
- explicit boundary conditions,
- notes on whether the existing test suites covered those classes (updated to `Yes (improved)` for every row that the improved suite closes).

This step was important because several high-coverage tasks still had obvious blind spots before the improved tests were added. Examples include:

- empty or whitespace-only parsing inputs,
- empty substring semantics,
- exact-threshold numeric comparisons,
- singleton and two-element collection boundaries,
- non-letter case-flip behavior,
- perfect-square composite numbers in primality checks.

The main conclusion of the black-box phase is that dataset base tests are strong but not uniformly complete. Their biggest weakness is not low branch coverage; it is the under-specification of invalid and boundary classes that are easy to miss when only happy-path prompt examples are used.

VIII. Refactoring Loop

The refactoring loop was triggered when a normalized solution either failed compilation, produced incorrect results on the dataset base tests, or exposed a structural error that would invalidate later coverage analysis. The repaired

tasks and their main issues were:

- `Java/10`: palindrome suffix handling;
- `Java/12`: empty-list and first-longest behavior;
- `Java/17`: music token parsing;
- `Java/19`: numeral-word sorting;
- `Java/23`: malformed duplicate method fragments in the normalized output;
- `Java/27`: incorrect case-flipping behavior;
- `Java/39`: inefficient or incorrect prime Fibonacci search.

These repairs confirm an important practical point: in LLM-assisted development, correctness often depends less on the initial generation and more on how quickly failures are surfaced and fixed by the surrounding workflow.

IX. Continuous Integration

Phase 1 ships a dedicated GitHub Actions workflow (`.github/workflows/phase1-ci.yml`) that mirrors the local pipeline on an `ubuntu-latest` runner:

1. installs Temurin JDK 21, Python 3.11 and PowerShell Core;
2. downloads JUnit 6 and JaCoCo 0.8.12 (cached between runs);
3. verifies that every Java source file is free of a UTF-8 BOM;
4. executes the base and improved suites through `scripts/python/run_base_coverage.py`;
5. asserts that every `(task, model)` pair reports `compile_success=true` and `junit_success=true`;
6. uploads `results/*.csv` and the per-run artefacts as a CI artefact.

Any regression in either suite therefore breaks the pull request.

X. Literature Review

The recent literature supports the structure used in this phase. ChatUniTest shows that raw LLM generations need validation and repair. The Meta TestGen-LLM paper shows that industrial adoption depends on strict filtering rather than trusting raw completions. Yang et al. demonstrate that prompt structure and open-source model choice materially affect Java unit test generation outcomes. Panta et al. show that hybrid static-plus-dynamic feedback can improve coverage on complex methods. The 2026 systematic literature review confirms that coverage is still the dominant metric in this area, but it also warns that integration complexity and robustness remain open issues. Together, these papers justify the semi-agentic, coverage-aware, mutation-augmented workflow used in this repository.

XI. Discussion

DeepSeek was slightly stronger than Qwen on aggregate branch coverage, both before and after the improved tests. Qwen still produced very strong final results, but it needed more residual branch follow-up on a small number of parsing and search-heavy tasks. The difference is not large enough to claim that one model is categorically superior for all selected problems; instead, the results suggest that both public lightweight models are viable under a disciplined validation workflow.

Another important result is that base-test success and branch coverage are strongly related but not interchangeable. Both models reached perfect base-test success, yet manual black-box analysis and mutation-based guardrails still

found meaningful missing classes. In other words, a suite can be "good enough to pass the benchmark" while still leaving real specification boundaries weakly documented.

XII. Threats to Validity

The main threats to validity are:

- prompt selection bias, since only 30 tasks from the dataset were used;
- difficulty labeling subjectivity, even though a balanced rubric was applied;
- local-model randomness and quantization effects;
- normalization heuristics, which may help or harm raw output quality depending on the generation style;
- dependence on HumanEval-X base tests and examples, which may not fully represent real-world Java project complexity;
- the mutation set is hand-picked (not machine-generated) and must be interpreted as a lower bound on the improved suite's robustness;
- group-of-two execution may cause review blind spots; we partially offset this by rotating code reviewers task-by-task.

Another threat is that some black-box judgments remain analyst-driven. Even when coverage numbers are objective, the decision that a missing case is important still depends on tester interpretation.

XIII. Conclusion

Phase 1 shows that public lightweight coder models can solve a substantial HumanEval-X Java subset with high correctness and high branch coverage when they are placed inside a careful engineering workflow. The decisive factors were not just model choice, but also normalization, test execution, coverage measurement, black-box analysis, mutation-based guardrails and repair when failures were exposed. The final Phase 1 artefact is therefore not simply a collection of generated solutions; it is a documented evaluation pipeline that can be extended directly into Phase 2 integration testing.

Phase 2 will lift three of Phase 1's constraints:

1. replace the PowerShell-driven JUnit runner with a Maven profile so that PITest can produce machine-measured mutation scores;
2. integrate the 30-prompt solutions into the BookScan case study to measure end-to-end quality under realistic system calls;
3. automate prompt variation so that we can study the sensitivity of the two models to prompt engineering in a controlled manner.

The hand-crafted mutants and equivalence classes from Phase 1 will be retained as behavioural guardrails even after PITest is turned on; they describe intent, not coverage.

Acknowledgement

- GitHub repository URL: `https://github.com/mustafaerenkoc44/BLG-475E-Term-Project`
- The two-person group split the Phase 1 workload along two complementary tracks. Both authors contributed equally and cross-reviewed each other's deliverables before they were merged.
- Group member roles and workload distribution:

- Mustafa Eren KOÇ (150190805) - engineering and automation track: repository scaffolding, GitHub Actions CI workflow, PowerShell + Python 3.11 driver layer, local `llama.cpp` execution pipeline, dataset-to-JUnit adapters, JaCoCo automation and result CSV aggregation, repair-loop driver, Git commit discipline.
- Onat Barış ERCAN (150210075) - quality and analysis track: 30-prompt selection rationale and easy / moderate / hard categorisation, per-task equivalence-partitioning and boundary-value documents, test-smell audit, hand-crafted mutation-operator catalogue and `improvedMutation...` JUnit method authoring, five-paper literature review, IEEE journal manuscript.

References for the Literature Section

1. Zhuokui Xie, Yinghao Chen, Chen Zhi, Shuiguang Deng, Jianwei Yin. ChatUniTest: a ChatGPT-based automated unit test generation tool. CoRR abs/2305.04764, 2023.
2. Nadia Alshahwan, Jubin Chheda, Anastasia Finogenova, Beliz Gokkaya, Mark Harman, Inna Harper, Alexandru Marginean, Shubho Sengupta, Eddy Wang. Automated Unit Test Improvement using Large Language Models at Meta. FSE 2024 Companion Proceedings, 2024.
3. Lin Yang, Chen Yang, Shutao Gao, Weijing Wang, Bo Wang, Qihao Zhu, Xiao Chu, Jianyi Zhou, Guangtai Liang, Qianxiang Wang, Junjie Chen. On the Evaluation of Large Language Models in Unit Test Generation. ASE 2024, 2024.
4. Sijia Gu, Noor Nashid, Ali Mesbah. LLM Test Generation via Iterative Hybrid Program Analysis. CoRR abs/2503.13580, accepted to ICSE 2026.
5. Murat Tasarsu, Ahmet Vedat Tokmak, Cagatay Catal. Test case generation using large language models: a systematic literature review. Cluster Computing, 2026.